

To help you transition from your previous medical chatbot to your advanced trackside **WEC & IMSA Cognitive Strategy and Diagnostics Assistant**, we will build out **Task 3 (The Physics Engine)** and **Task 4 (The Unified Streamlit UI App)**.

Running this stack locally protects your proprietary manufacturer telemetry and is fully compliant with AI Kamel Systems' strict data policy.

Make sure your directories are structured as follows in VS Code:

```
trackside_app/
├── physics/
│   ├── init.py
│   ├── fuel_burn.py
│   └── tire_deg.py
├── data/
├── manuals/          # Put your technical PDFs and rulebooks here
├── trackside_timing.db # Your local SQLite timing database
└── app.py            # Streamlit front-end
```

Step 1: Coding the Physics Engine (physics/fuel_burn.py)

As an endurance race stint progresses, the fuel mass inside the tank decreases linearly. Since weight directly dictates lateral and longitudinal grip, a lighter vehicle inherently achieves faster lap times.

To isolate and track true tire degradation over a stint, we must remove this fuel burn-off effect using our linear correction formula :

$$T_{\text{corrected}} = T_{\text{actual}} - \alpha_{\text{fuel}} \cdot \left(M_{\text{initial}} - M_{\text{vehicle}}(t) \right)$$

Create physics/fuel_burn.py and implement this logic:

Python

```
# physics/fuel_burn.py

def calculate_vehicle_mass(initial_mass_kg: float, lap: int, fuel_burn_per_lap_kg: float) -> float:
    """
    Calculates the instantaneous vehicle mass at the start of a given lap.

    Formula:  $M_{\text{vehicle}}(t) = M_{\text{initial}} - (t * m_{\text{dot\_fuel}})$ 
    """
    return max(0.0, initial_mass_kg - (lap * fuel_burn_per_lap_kg))

def calculate_fuel_corrected_time(
    actual_time_s: float,
    initial_mass_kg: float,
    current_lap: int,
    fuel_burn_per_lap_kg: float,
    alpha_fuel: float = 0.03
) -> float:
    """
    Corrects the actual lap time to isolate mechanical and tire grip.
    Normalizes the time to show what the pace would be at zero-fuel load.

    Formula:  $T_{\text{corrected}} = T_{\text{actual}} - \alpha_{\text{fuel}} * [M_{\text{initial}} - M_{\text{vehicle}}(t)]$ 

    Parameters:
    - actual_time_s (float): The raw recorded lap time in seconds.
    - initial_mass_kg (float): Starting mass of the car (wet weight) in kg.
    - current_lap (int): Laps completed in the current stint.
    - fuel_burn_per_lap_kg (float): Average fuel burn rate (e.g., 1.84 kg/lap).
    - alpha_fuel (float): Fuel penalty coefficient (standard approximation: 0.03s per kg).
    """
    current_mass = calculate_vehicle_mass(initial_mass_kg, current_lap,
    fuel_burn_per_lap_kg)
    mass_delta = initial_mass_kg - current_mass
    time_correction = alpha_fuel * mass_delta

    # We subtract the time correction since a lighter car is naturally faster
    corrected_time = actual_time_s - time_correction

    return round(corrected_time, 3)
```

(Note: An industry-standard rule of thumb is that every 10 kg of fuel burned yields a gain of 0.3 seconds in lap time, which corresponds to the α_{fuel} value of 0.03 s/kg used above).

Step 2: Coding the Non-Linear Tire Degradation Model

(physics/tire_deg.py)

Tire performance decays over its life-cycle through multiple phases.

1. **Warmup Phase:** The tire reaches its optimal core pressure and temperature, briefly increasing mechanical grip.
2. **Steady-State Phase:** Linear, predictable wear occurs (usually $0.02 \text{ to } 0.1 \text{ s/lap}$ depending on compound).
3. **Thermal Drop-Off Cliff:** Below a certain tread depth, heat rejection fails, causing core overheating. The car suddenly bleeds $3 \text{ to } 4 \text{ seconds per lap}$ within just a few laps :

$$\Delta_{\text{thermal}}(t) = c \cdot e^{k \cdot (t - t_{\text{threshold}})}$$

Crucially, in modern LMDh/LMH classes, telemetry might report plenty of physical rubber left on the tire even when core thermal grip has completely collapsed, creating a dangerous discrepancy. Create physics/tire_deg.py to capture this:

```
Python
# physics/tire_deg.py
import math

def predict_tire_degradation_penalty(
    lap: int,
    compound: str,
    track_temp_f: float,
    threshold_laps: int = 12
) -> float:
    """
    Predicts the performance drop-off (in seconds) due to tire wear.

    Formula:  $\Delta_{\text{thermal}}(t) = c \cdot e^{k \cdot (t - t_{\text{threshold}})}$ 
    """
    comp = compound.lower()

    # Configure compound-specific coefficients
```

```

if comp == "soft"
    linear_rate = 0.08 # Softs wear faster
    k = 0.65 # Aggressive exponential cliff curve
    c = 0.15 # Cliff base penalty scaling
elif comp == "medium"
    linear_rate = 0.05
    k = 0.45
    c = 0.10
else # "hard"
    linear_rate = 0.02
    k = 0.30
    c = 0.05

    threshold_laps += 5 # Hards last longer before hitting the cliff

# Track temperature penalty scaling (optimal is 95°F)
temp_factor = 1.0 - max(0.0, (track_temp - 95.0) / 30.0)
effective_linear_rate = linear_rate * temp_factor

# Phase 1: Warmup (First 2 laps show improvement, negative penalty)
if lap <= 2
    return 0.25 - 3 * lap

# Phase 2: Steady-State Linear Wear
base_wear_penalty = (lap - 2) * effective_linear_rate

# Phase 3: Non-Linear Thermal Cliff
cliff_penalty = 0.0
if lap > threshold_laps
    laps_past_threshold = lap - threshold_laps
    # Exponential curve: c * e^(k * t_diff)
    cliff_penalty = c * math.exp(k * laps_past_threshold) * temp_factor

return round(base_wear_penalty + cliff_penalty, 3)

```

Step 3: Coding the Unified Trackside Chatbot App (app.py)

Now, let's write your Streamlit front-end (app.py). This app serves as an advanced offline cockpit dashboard.

It replaces your previous vaccine RAG pipeline with a localized dual-engine framework:

1. **Technical Manual RAG Tool:** Uses LlamaIndex and a local model to parse nested schematics, regulations, and hybrid codes offline.

2. **Strategic Text-to-SQL Tool:** Queries your local SQLite database for lap times and telemetry.
3. **Physics Predictor Visualizer:** Plots fuel burn and tire degradation interactively.

```
Python
# app.py
import streamlit as st
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
from langchain_community.llms import Ollama
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# Import our custom physics modules
from physics.fuel_burn import calculate_fuel_corrected_time
from physics.tire_deg import predict_tire_degradation_penalty

st.set_page_config(page_title="WEC & IMSA Strategy Assistant" layout="wide"

# --- DATABASE SETUP ---
def execute_safe_query(sql_query: str) -> pd.DataFrame:
    forbidden = "drop" "delete" "insert" "update" "alter" "truncate"
    if any(k in sql_query.lower() for k in forbidden):
        st.error("🚫 Security Block: Destructive SQL query rejected.")
    return pd.DataFrame()

conn = sqlite3.connect("trackside_timing.db")
try
    df = pd.read_sql_query(sql_query, conn)
    return df
except Exception as e:
    st.error(f"Database Error: {e}")
    return pd.DataFrame()
finally
    conn.close()
```

```

# --- SIDEBAR CONFIGURATION ---
with st.sidebar:
    st.header("⚡ Trackside Configurations")
    series = st.selectbox("Championship Series")
    compound = st.selectbox("Tire Compound Set")
    track_temp = st.slider("Track Temperature (°F)" 60 140 95)
    stint_laps = st.number_input("Stint Laps Forecast" min_value=5 max_value=40 value=20)

    st.divider()
    st.subheader("System Status")
    st.success("Ollama Engine: Connected (Port 11434)")
    st.info("Database: trackside_timing.db active")

    st.title("🏎️ WEC & IMSA Cognitive Strategy & Diagnostics Assistant")
    st.markdown("----")

# --- TAB 1: OFF-LINE CHATBOT ---
tab_chat, tab_physics = st.tabs()

with tab_chat:
    st.subheader("Interactive Strategy & Diagnostics")

    # Initialize Local LLM via Ollama
    try:
        llm = Ollama(model="qwen2.5-coder:7b", base_url="http://localhost:11434")
    except Exception as e:
        st.error("Could not reach local Ollama server. Verify that Ollama is running in your background taskbar.")

    user_query = st.chat_input("Ask about strategy, competitor sector times, or request hybrid fault code troubleshooting...")

    if user_query:
        with st.chat_message("user"):
            st.write(user_query)

        with st.chat_message("assistant"):
            with st.spinner("Processing local engines..."):
                # Determine query intent: Diagnostics vs Database Timing
                routing_prompt = f"""
                You are a senior Race Strategy Engineer. Categorize the user's intent.
                If the query asks about manual instructions, hybrid codes, rules, or diagnostics, output

```

```

'RAG'.
    If the query asks to search lap times, sector times, positions, or ELO, output 'SQL'.
    Query: "{user_query}"
    Output exactly one word: 'RAG' or 'SQL'.
    """
    intent = llm.invoke(routing_prompt.strip()).upper()

    if "SQL" in intent:
        st.caption("📊 *Routing to: SQLite Database Timing Engine*")
        # Text-to-SQL logic
        sql_gen_prompt = f"""
        Given the SQLite table 'laps' with fields:
        series_code, class, driver_name, lap_time_s, s1_s, s2_s, s3_s, pit_time_s, track_temp_f,
        raining.
        Generate a clean SQL SELECT query to answer: "{user_query}".
        Respond with ONLY the raw SQL query, no markdown blocks, no format.
        """
        sql_query = llm.invoke(sql_gen_prompt.strip().replace("`", "`")).replace("sql", "")
        st.code(sql_query, language="sql")

        df_results = execute_safe_query(sql_query)
        if not df_results.empty:
            st.dataframe(df_results)
            # Natural language summarization of results
            summary = llm.invoke(f"Summarize these timing database results for the engineer: {df_results.to_string()}")
            st.write(summary)
        else:
            st.warning("No timing records matched your query.")

    else:
        st.caption("📖 *Routing to: Advanced Technical Manual RAG*")
        try:
            # Direct RAG over any PDFs in data/manuals/
            documents = SimpleDirectoryReader("data/manuals").load_data()
            index = VectorStoreIndex.from_documents(documents)
            query_engine = index.as_query_engine()
            response = query_engine.query(user_query)
            st.write(str(response))
        except Exception as e:
            st.info("Place technical PDFs (e.g. Bosch MGU/MCU troubleshooting manuals) inside 'data/manuals' to enable advanced diagnostic RAG.")
            st.write(llm.invoke(user_query))

```

```

# --- TAB 2: PHYSICS PREDICTIONS ---
with tab_physics:
    st.subheader("Stint Pace Decay & Degradation Forecast")

    # Generate simulation curves
    laps_seq = list(range(1, stint_laps + 1))

    # Example Baseline reference pace (Daytona Hypercar baseline)
    ref_lap_time = 95.0 # seconds
    initial_fuel = 100.0 # kg
    fuel_burn_rate = 1.84 # kg per lap

    raw_laps =
    fuel_corrected_laps =
    tire_penalties =
    combined_pace =

    for i in laps_seq:
        # Simulate tire penalty
        t_pen = predict_tire_degradation_penalty(i, compound, track_temp)
        tire_penalties.append(t_pen)

        # Simulated actual times (getting heavier or lighter depending on fuel drop)
        # Actual lap gets lighter, saving time, but tire wears out costing time.
        simulated_actual = ref_lap_time + t_pen - 0.03 * (initial_fuel - (initial_fuel - (i *
fuel_burn_rate)))
        raw_laps.append(simulated_actual)

    # Apply physics correction formula to clean out fuel burn advantage
    corrected = calculate_fuel_corrected_time(simulated_actual,
initial_mass_kg=initial_fuel, current_lap=i, fuel_burn_per_lap_kg=fuel_burn_rate)
    fuel_corrected_laps.append(corrected)

    # Plotting
    fig, ax = plt.subplots(figsize=(10, 4))
    ax.plot(laps_seq, raw_laps, label="Simulated Actual Lap Times (Masked by Fuel Burn-off)",
color="red", linestyle="--", marker="o")
    ax.plot(laps_seq, fuel_corrected_laps, label="Fuel-Corrected Pace (Pure Mechanical/Tire
Degradation)", color="green", linewidth=2.5, marker="s")

    # Highlight critical tire cliff zone
    threshold = 12 if compound.lower() == "soft" else 17 if compound.lower() == "medium" else
22

```

```

if stint_laps > threshold
    ax.axvspan(threshold, stint_laps, color='yellow', alpha=0.2, label='Tire Cliff Operating Window')

ax.set_xlabel("Stint Lap Number")
ax.set_ylabel("Lap Time (Seconds)")
ax.set_title(f"Endurance Stint Progression on {compound} Compound")
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend()

st.pyplot(fig)

# Visual metrics cards
col1, col2, col3 = st.columns(3)
with col1:
    st.metric(label="Simulated Stint End Penalty", value=f"{tire_penalties[-1]:.2f} s")
with col2:
    st.metric(label="Predicted Tire Cliff Threshold", value=f"Lap {threshold}")
with col3:
    st.metric(label="Stint Average Pace", value=f"{pd.Series(fuel_corrected_laps).mean():.3f} s")

```

Step 4: Run the System Locally

With both packages active inside your conda environment, run:

```

Bash
streamlit run app.py

```

This launch starts your trackside UI instantly and completely offline. Place all your technical WEC or IMSA regulations and bulletins inside the `/data/manuals/` folder, open up the app browser view, and your pit-wall engineers can ask questions, analyze tire drops, and debug hybrid issues on demand.